



Reinforcement Learning

Многоагентное обучение
и кооперация агентов



Проверить, идет ли запись

Меня хорошо видно && слышно?



Ставим "+", если все хорошо
"-", если есть проблемы



Тема вебинара

Reinforcement Learning

Многоагентное обучение

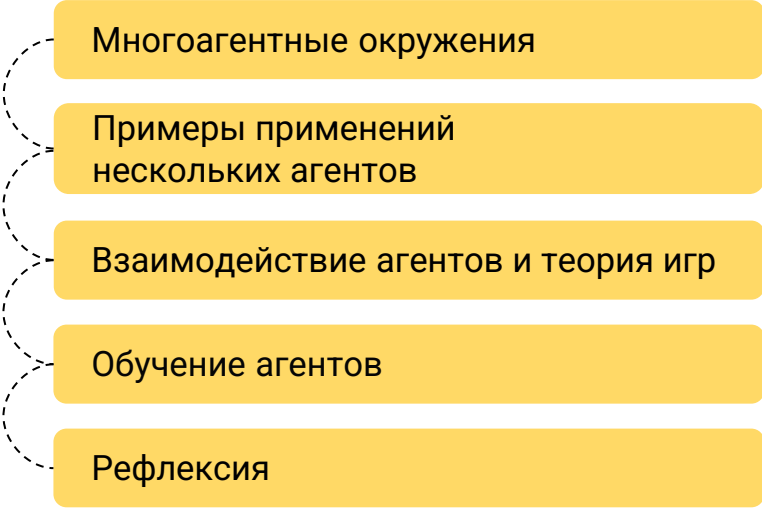
Андрей Канашов

Team Lead Data Scientist в ПИК



- Ценообразование и тарификация
- Рекомендательные системы
- Прогнозирование ключевых метрик
- Анализ клиентского поведения

Маршрут вебинара



Многоагентные окружения

Примеры применений
нескольких агентов

Взаимодействие агентов и теория игр

Обучение агентов

Рефлексия

Цели вебинара

К концу занятия вы узнаете

1. Чем примечательны многоагентные окружения
2. Когда от мультиагентных подходов будет польза, а когда нет
3. Как применяется теория игр в решении задач MARL
4. Какие идеи лежат в основе распространённых MARL алгоритмов

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
В чате учебной группы



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом



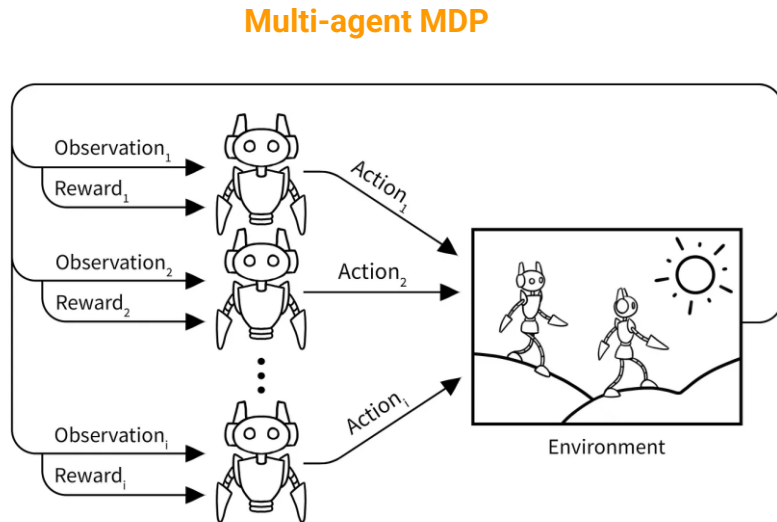
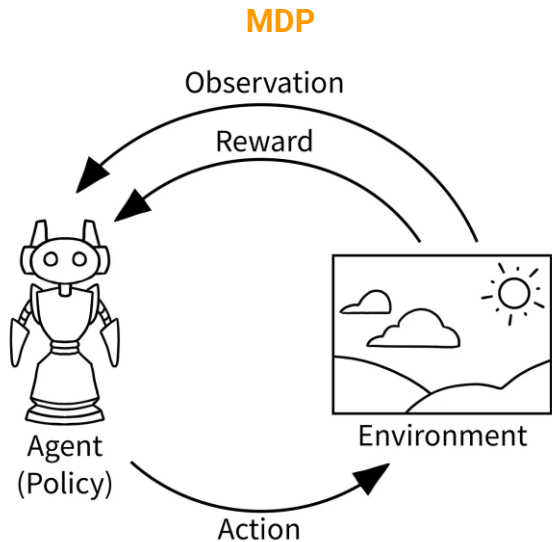
Документ



Ответьте себе или
задайте вопрос

Многоагентные окружения

Ещё один вид MDP



Проблемы многоагентных сред

- **Нестационарность среды** - другие агенты могут менять среду и нарушать её стационарность
- **Координация и кооперация агентов** - агенты вынуждены согласовывать свои действия, чтобы не мешать друг другу или чтобы достигать общей цели
- **Частично наблюдаемая среда** - обычно каждый агент получает индивидуальные наблюдения
- **Сложно справедливо распределять награду** - помимо отложенных наград добавляется проблема определить точный вклад каждого агента в полученной награде





В каких случаях полезно вводить
несколько агентов, а в каких нет?

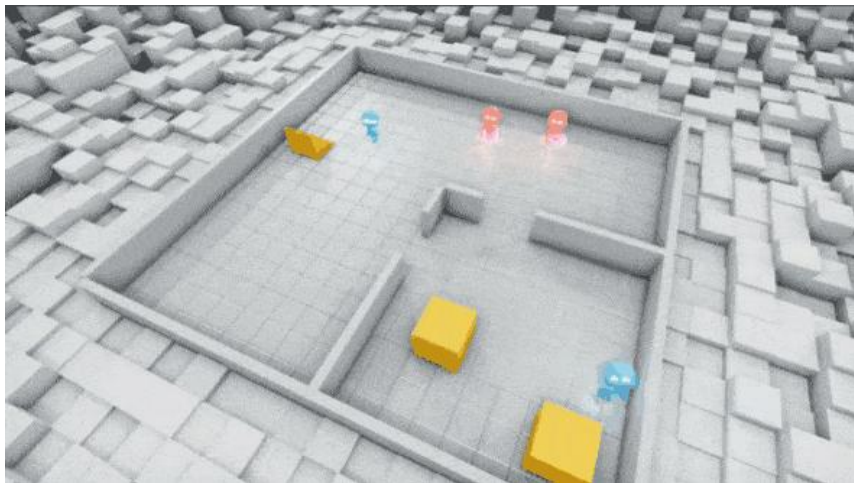
Примеры применений нескольких агентов

Противостояние обучаемых агентов

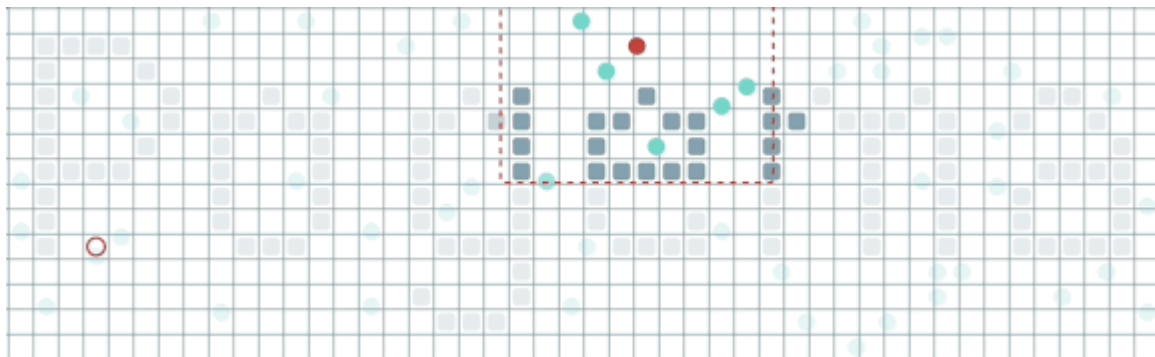
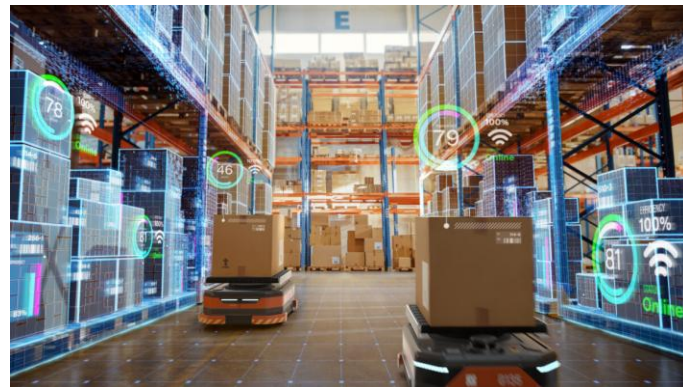
Симметричные окружения



Несимметричные окружения

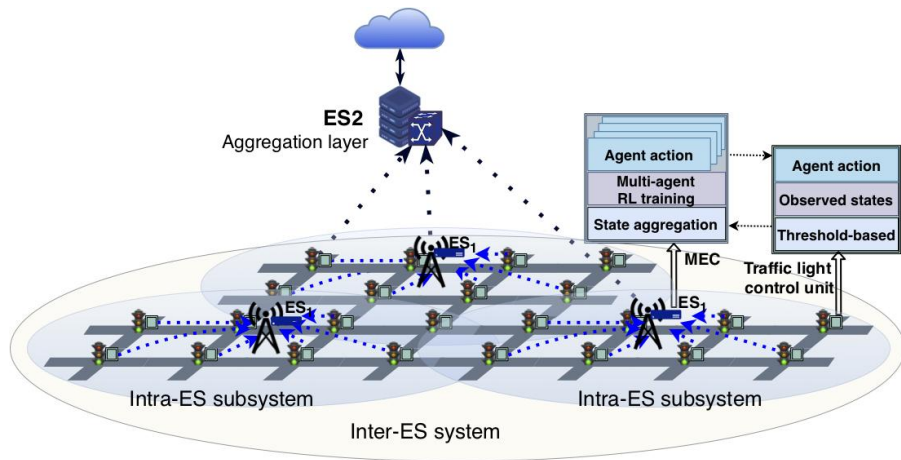


Управление роботами на складах

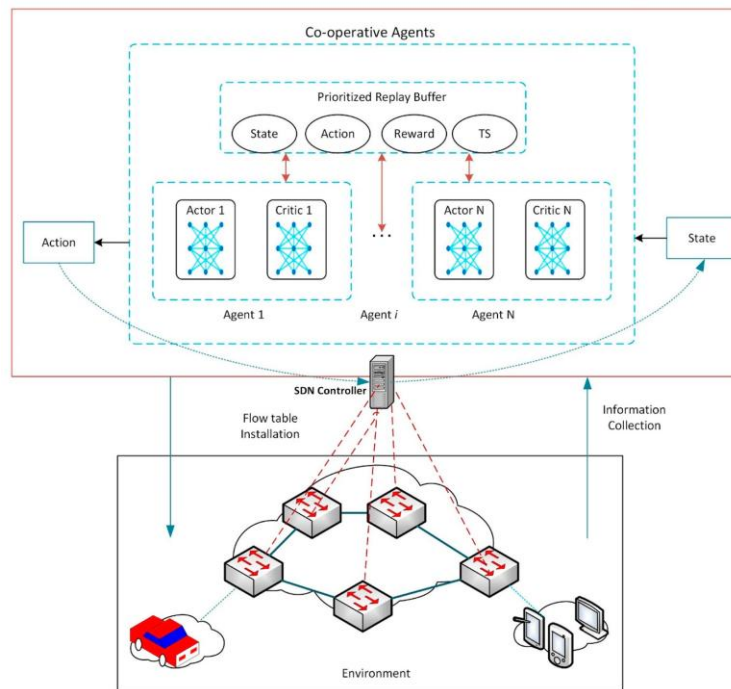


Маршрутизация трафика

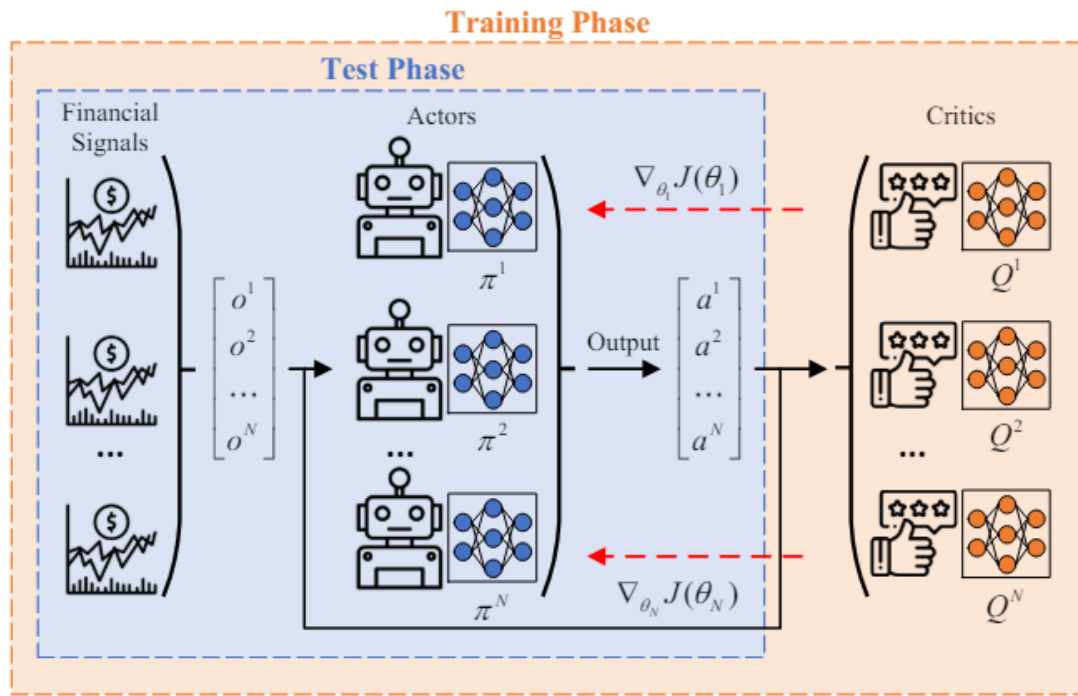
Дорожный трафик



Интернет трафик



Торговля на фондовых и валютных рынках



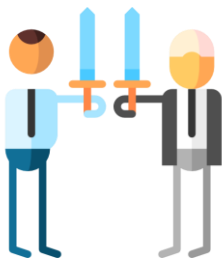
Взаимодействие агентов и теория игр

Виды игрового взаимодействия агентов

Независимое
взаимодействие



Кооперация



Конкуренция

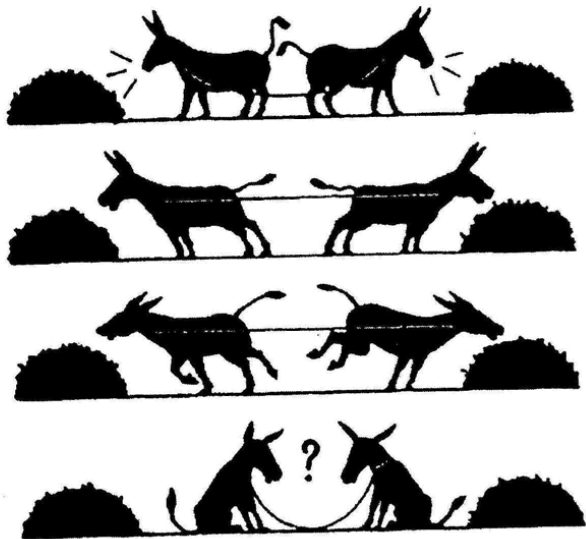


Смешанное
взаимодействие

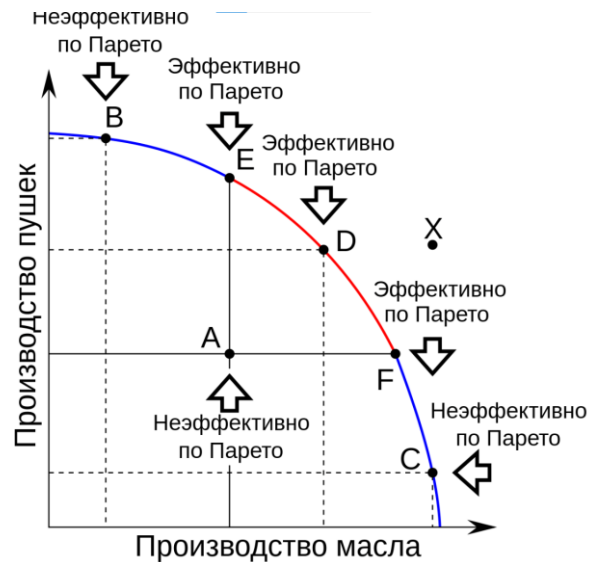
Дилемма заключённого

		Ваш сообщник	
		Хранит молчание	Доносит
Вы	Хранит молчание	Оба получают 2 года тюрьмы	Вы получаете 5 лет тюрьмы; его освобождают
	Доносит	Он получает 5 лет тюрьмы; вас освобождают	Оба получают 4 года тюрьмы

Равновесие Нэша и Парето-оптимальность



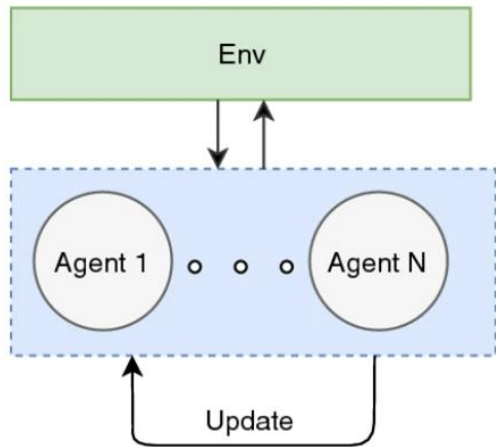
Равновесие по Нэшу - когда ни один игрок не может получить дополнительную выгоду от своих измененных действий, если другие игроки продолжают свои стратегии.



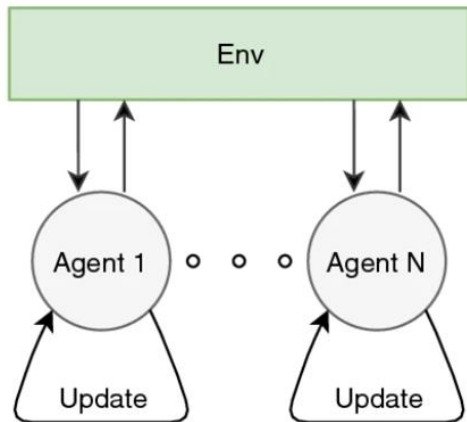
Эффективность по Парето - такое состояние системы, при котором ни один показатель системы не может быть улучшен без ухудшения какого-либо другого показателя.

Алгоритмы для мультиагентного обучения

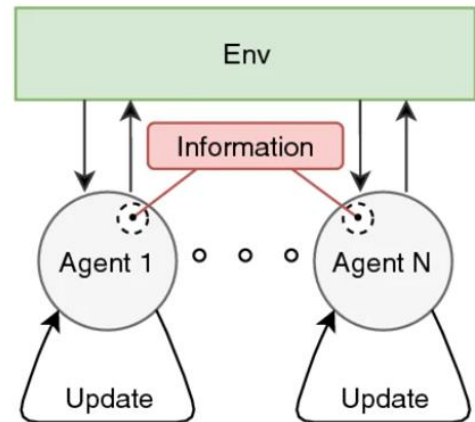
Схемы обучения агентов



Centralized training
with centralized execution
(CTCE)



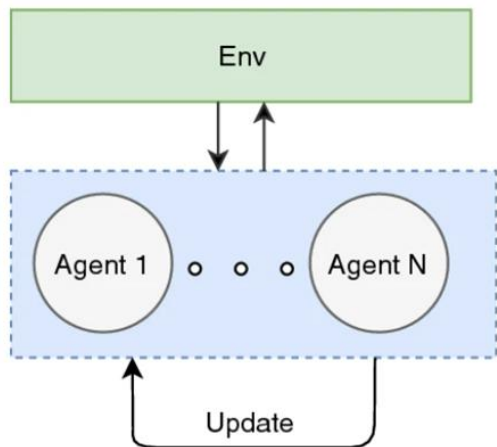
Decentralized training
with decentralized execution
(DTDE)



Centralized training
with decentralized execution
(CTDE)

Централизованное обучение

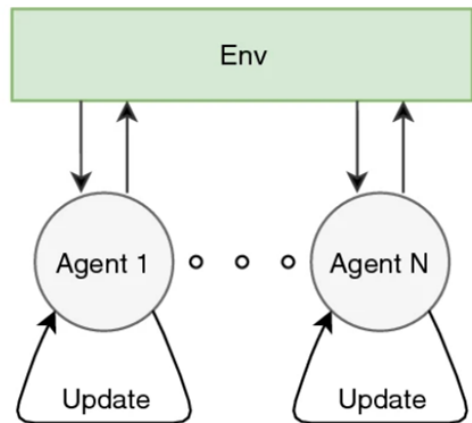
Centralized training
with centralized execution
(CTCE)



- Полностью централизованный сеттинг возможен для агентов, например, оптимизирующих одну и ту же функцию награды, находящихся в **кооперации**.
- Полная централизация означает, что для построения системы из нескольких кооперирующихся агентов все они рассматриваются **как единый агент**. Наблюдение есть совокупность наблюдений всех агентов, действия — набор действий для каждого из агентов, и тогда по сути задача сведена к «одноагентной» ситуации.
- В большинстве случаев такой вариант не годится по самой постановке задачи: наличие единого **«центрального сервера»**, который бы принимал решения за всех агентов в среде, может быть просто-напросто очень дорогим. Хотя бы потому что агентов может быть много, и пространство состояний и, главное, действий, взорвётся.
- В большинстве случаев хочется, чтобы агенты принимали решения на основе только имеющейся у них информации.

Децентрализованное обучение

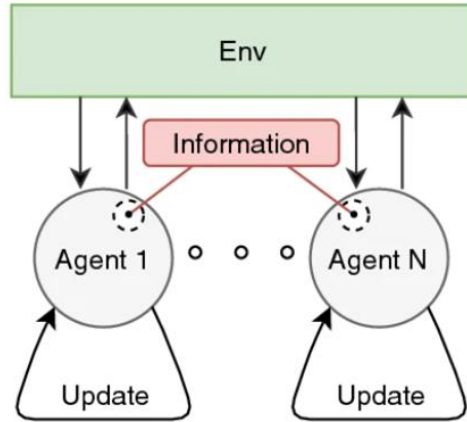
Decentralized training with decentralized execution (DTDE)



- Децентрализованный сеттинг означает, что у агентов нет какой-либо общего «сервера» с общими данными, в том числе во время обучения.
 - Простейший подход мультиагентного RL в рамках такого сеттинга — завести для каждого агента свою стратегию $\pi_i(a | s)$ и функцию оценки $Q_i(s, a_i)$, которая будет зависеть только от действия i -го агента, и оптимизировать их с опыта данного агента любыми обычными RL алгоритмами.
 - Тот факт, что в среде **действуют другие агенты, игнорируется**. Важный момент: если другие агенты не обучаются (не изменяются с течением времени), то такой подход обучения i -го агента корректен.
-
- Это означает, что если в среде действует другой агент, использующий стационарную стратегию, среда остаётся стационарной и вся теория остаётся справедлива. Однако, если в среде действует другой агент, который обучается (и, значит, меняет свою стратегию с течением времени), то для прочих агентов это означает, что постоянно изменяются «законы физики» окружающей среды, и мы под изученную теорию подпадать перестаём.
 - В нестационарных средах естественно использовать on-policy обучение. Запуск какого-нибудь on-policy алгоритма с игнорированием факта присутствия других обучающихся агентов обычно оказывается серьёзным бэйзлайном.

Частичное централизованное обучение

Centralized training
with decentralized execution
(CTDE)



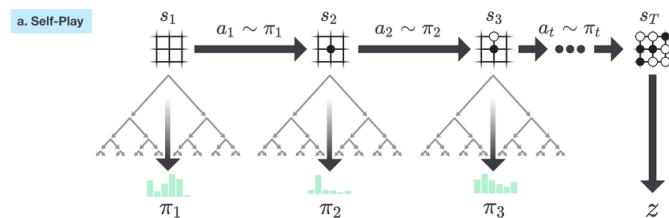
- У каждого из кооперирующихся агентов хранится и обучается персональная стратегия, но функция ценности $Q(s, a_1, a_2 \dots)$ хранится и обучается на общем для всех агентов «сервере».
- Важно, что эта оценочная функция нужна в on-policy алгоритмах только для обучения, и при использовании полученных стратегий «сервер» не потребуется. Это означает, что по окончании обучения в таком сеттинге для использования стратегии никакого центрального сервера не понадобится.
- Возможен **parameter sharing**: если задача симметрична для каких-то агентов, то веса их стратегий считаются общими. Градиенты для их обучения тогда вычисляются на общем сервере. Далее во время использования агенты используют одну и ту же модель для принятия решений.
- В CTDE при желании обучить колонию из 100 одинаковых муравьёв, понадобится лишь **одна модель**. В децентрализованном сеттинге, подразумевается, что каждый из муравьёв должен как-то сам обучиться, и тогда у каждого из сотни получится в итоге своя модель.



Что делать, если количество
управляемых агентов в среде
будет изменяться?

Self-play

Self-play

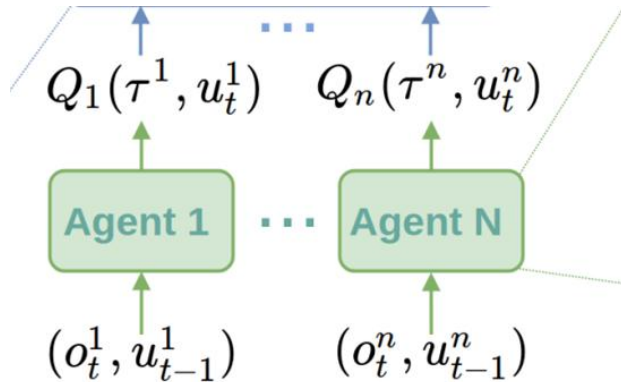


- В симметричных антагонистических играх обучение на опыте игр с самим собой оказалось мощным инструментом. Этот подход называют self-play, и заключается он в том, что за игроков играет **одна и та же стратегия** (такие игры обычно симметричны, и поэтому можно считать, что применяется parameter sharing).
- Собранный опыт со стороны каждого игрока можно использовать для обучения, и он обладает одним очень ценным свойством: **сбалансированностью**. На какой бы стадии обучения ни был агент, играет ли он на уровне гранд-мастера или новичка, при игре с самим собой в половине случаев он «выигрывает», в половине — «проигрывает». Разница в сигнале всегда даёт возможность дальнейшего улучшения.
- В режиме self-play можно запустить любые алгоритмы обучения: как model-free, так и model-based. В частности, в model-based алгоритмах вроде AlphaGo и MuZero в ходе планирования можно при построении дерева ходы за оппонента проводить при помощи тех же моделей, но не максимизировать, а минимизировать награду.

QMIX

QMIX

Рассмотрим кооперативный сеттинг, где все агенты максимизируют одну и ту же функцию награды

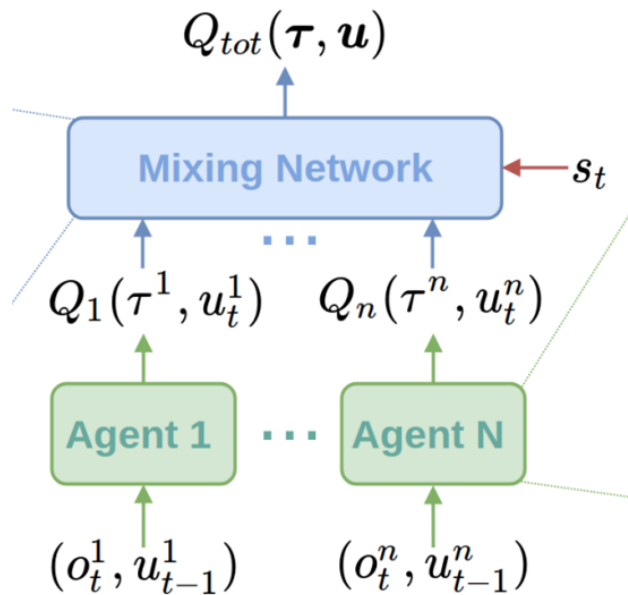


Будущую кумулятивную награду после выполнения набора действий $\mathbf{a}$ в состоянии s для набора политик $\pi(\mathbf{a} | s)$ обозначим как $Q^{\text{tot}}(s, \vec{a})$ скаляр.

Попробуем построить алгоритм в рамках парадигмы CTDE, то есть обучить «на центральном сервере» стратегии $\pi(\mathbf{a} | s)$, которые смогут дальше действовать в среде без использования подобного сервера. Допустим, мы знаем $Q^{\text{tot}}(s, \vec{a})$. Тогда оптимально выбрать действия $\underset{\vec{a}}{\operatorname{argmax}} Q^{\text{tot}}(s, \vec{a})$: однако есть две проблемы:

1. Если агентов много, поиск такого аргмаксимума может стать слишком сложной задачей, ровно как и моделирование такой оценочной функции.
2. Во-вторых, в рамках CTDE в частично наблюдаемом сеттинге каждый агент на вход вместо s на самом деле получает своё собственное наблюдение o_i и не знает наблюдения других агентов, когда Q^{tot} принимает на вход наблюдения всех агентов.

QMIX



Смешивающая сеть (**Mixing Network**)

$$Q^{\text{tot}}(s, \vec{a}) \approx Q_{\phi}^{\text{tot}}(Q^1(s, a^1), Q^2(s, a^2), \dots, Q^N(s, a^N), s)$$

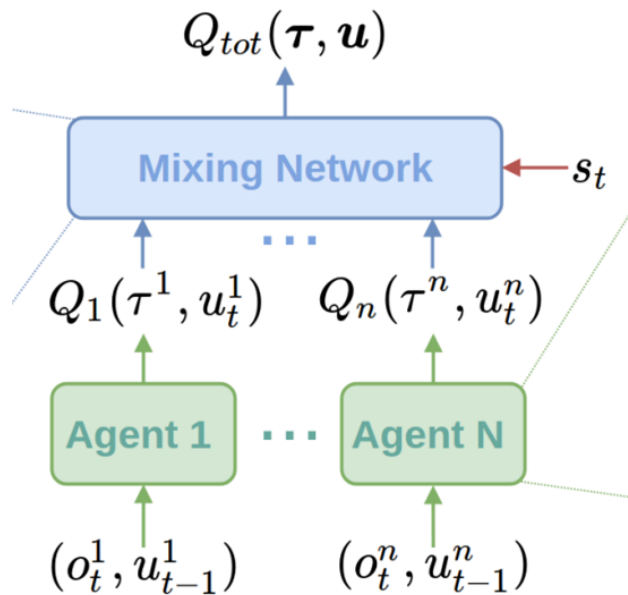
Рассмотрим самый простой пример смешивающей сети, не использующей параметров ϕ вовсе:

$$Q_{\phi}^{\text{tot}}(s, \vec{a}) := \sum_i Q^i(s, a^i)$$

Свойство КОНСИСТЕНТНОСТИ:

$$\operatorname{argmax}_{\vec{a}} Q^{\text{tot}}(s, \vec{a}) = \begin{bmatrix} \operatorname{argmax}_{a^1} Q^1(s, a^1) \\ \operatorname{argmax}_{a^2} Q^2(s, a^2) \\ \vdots \\ \operatorname{argmax}_{a^N} Q^N(s, a^N) \end{bmatrix}$$

QMIX



Смешивающая сеть консистентна, если

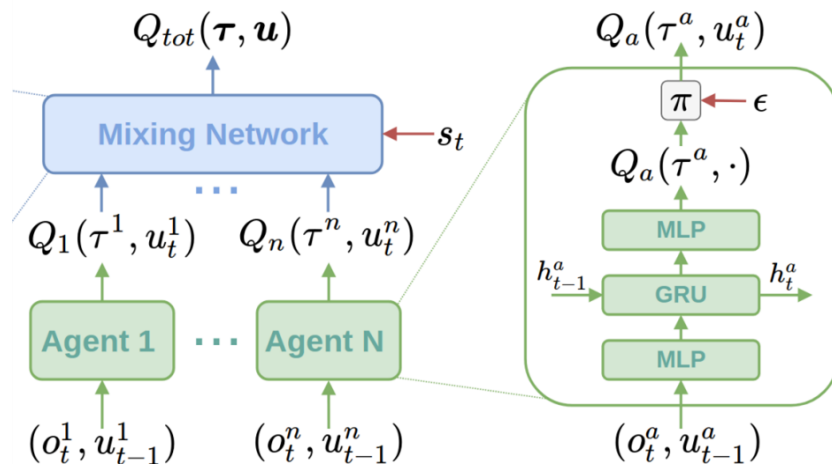
$$\frac{dQ_{\phi}^{\text{tot}}}{dQ^i} \geq 0$$

то есть для всех состояний s и при любых значениях параметров ϕ награда команды монотонно зависит от значений псевдооценочных функций агентов $Q_i(s, a_i)$

следовательно максимизация всех аргументов монотонной функции максимизирует саму функцию.

QMIX

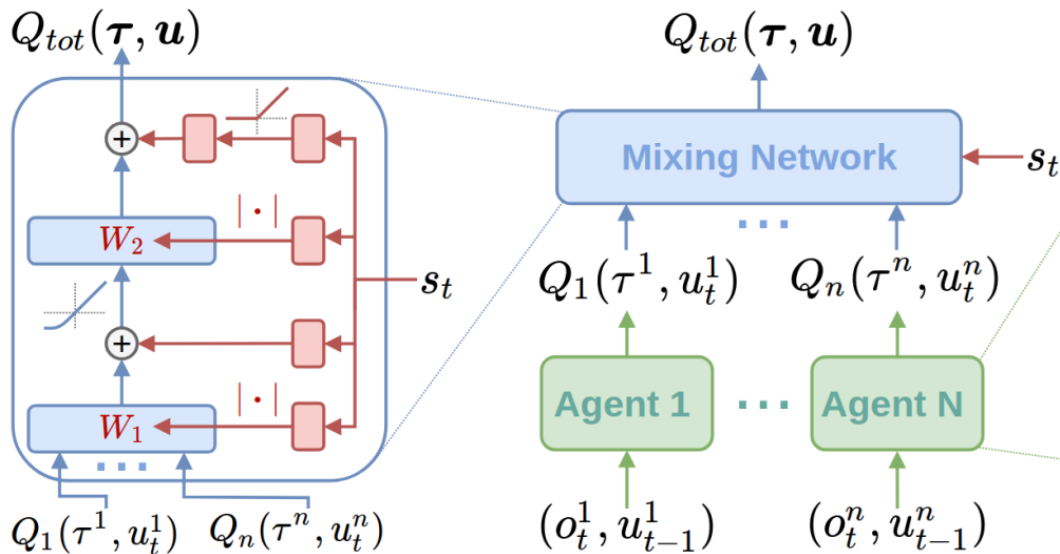
- В частично наблюдаемом сеттинге $Q_i(s, a_i)$ вместо s принимает на вход наблюдение i -го агента o_i и моделируется рекуррентной сетью.
- $Q_i(s, a_i)$ не является оценочной функцией и не имеет смысла будущей кумулятивной награды. Это лишь некоторая промежуточная вспомогательная величина, через которую выражается: $Q^{\text{tot}}(s, \vec{a})$ награда «всей команды».



- Консистентность** - каждому агенту не нужно знать наблюдения других агентов или обращаться к центральному серверу для выбора оптимального действия, он берёт лишь значение своей псевдооценочной функции $Q_i(s, a_i)$ и считает аргмаксимум по нему.

QMIX

Пусть нейросеть полносвязная, состоит из чередования линейных слоёв и монотонных функций активаций, и все параметры нейросети ϕ_i (кроме, возможно, смещений в линейных слоях) неотрицательны.

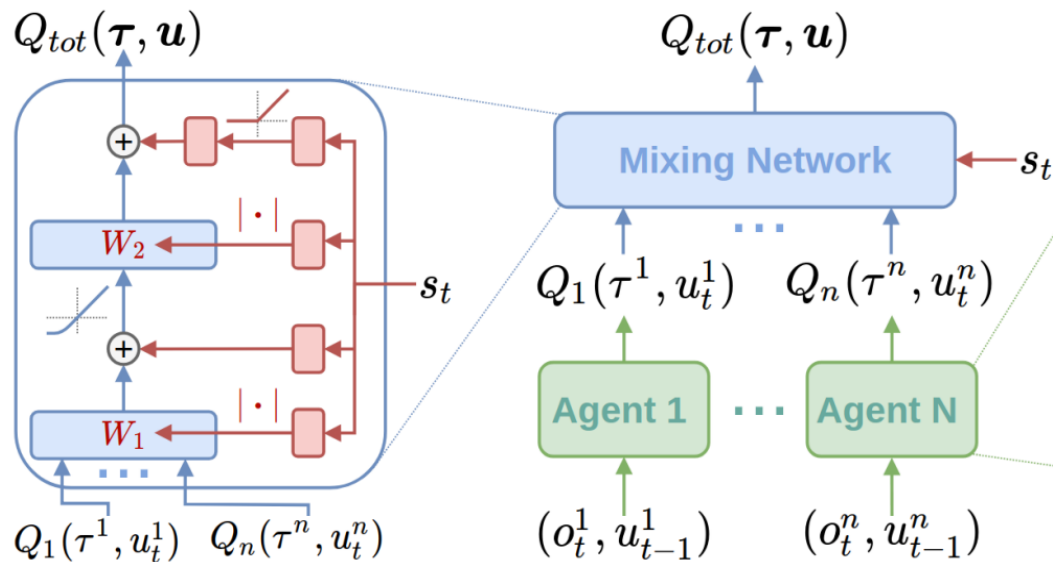


Композиция монотонных преобразований монотонна; линейная комбинация с положительными весами (и произвольными смещениями) также монотонно не убывает как функция от входов.

QMIX

Гиперсеть (hypernetwork) называется нейросеть, выдающая веса для другой нейросети.

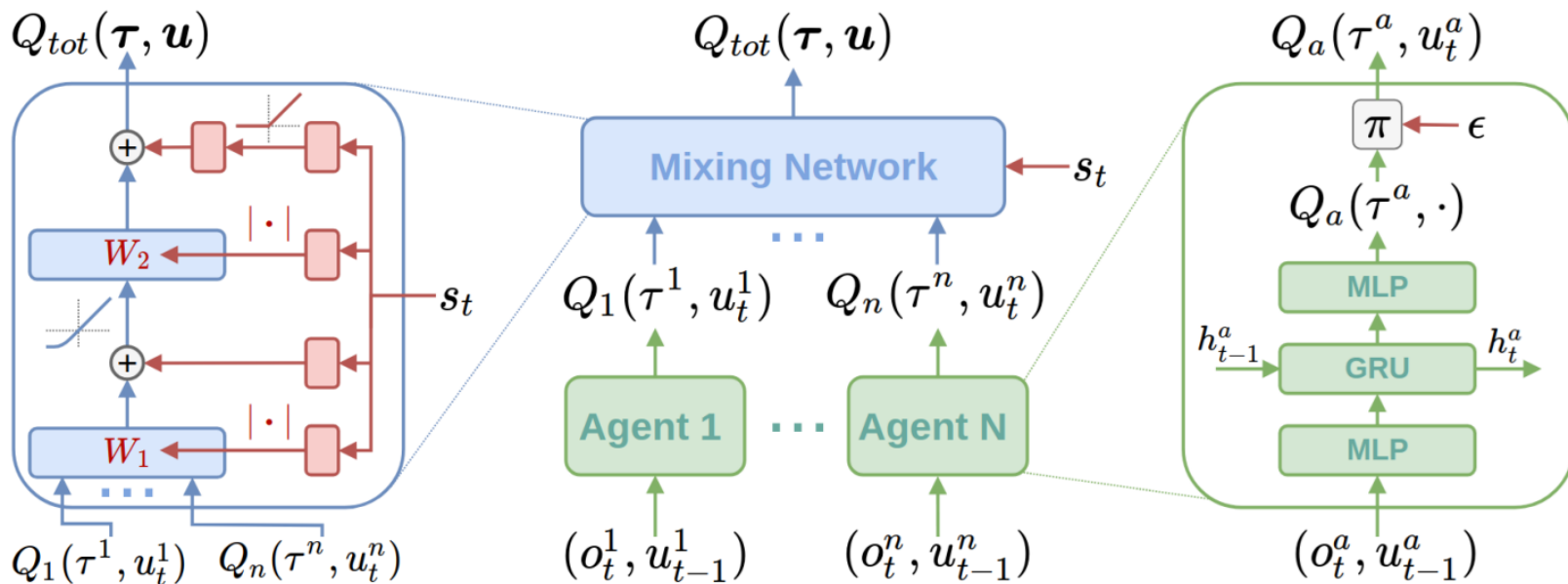
Мы заведём гиперсеть $\phi(\mathbf{s})$, которая для данного состояния $\mathbf{s}$ будет с параметрами θ выдавать на центральном сервере веса для смешивающей сети.



Получим, что в каждом состоянии $\mathbf{u}$ у нас есть какое-то своё монотонное преобразование псевдооценочных функций каждого агента в оценочную функцию команды.

QMIX

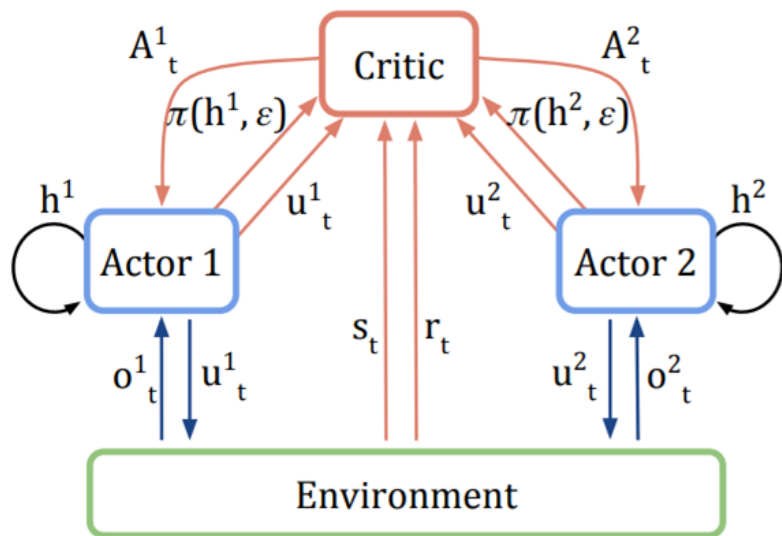
Monotonic Value Function Factorisation



COMA

COMA

Counterfactual Multi-Agent Policy Gradients

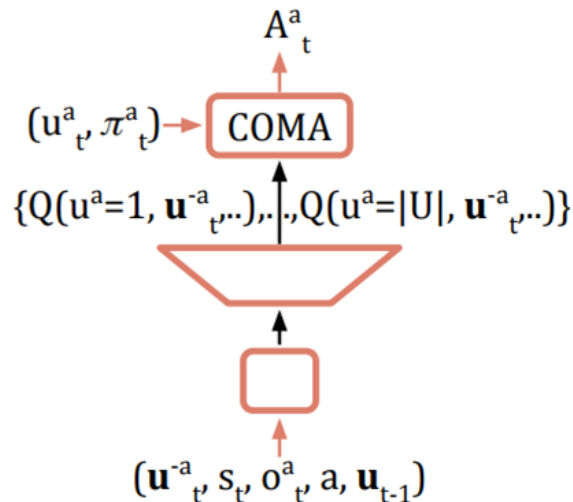


Подход на основе актор-критика для многоагентных систем, который позволяет обучить полностью централизованную функцию ценности действий-состояний и использовать ее для оптимизации децентрализованных политик.

Использует централизованного критика для обучения децентрализованных акторов для каждого агента, оценивая контрфактическую функцию преимущества для каждого.

Используется контрфактический базис, который маргинализирует действие одного агента, сохраняя действия других агентов неизменными, а централизованное представление критика позволяет эффективно вычислять контрфактический базис.

COMA



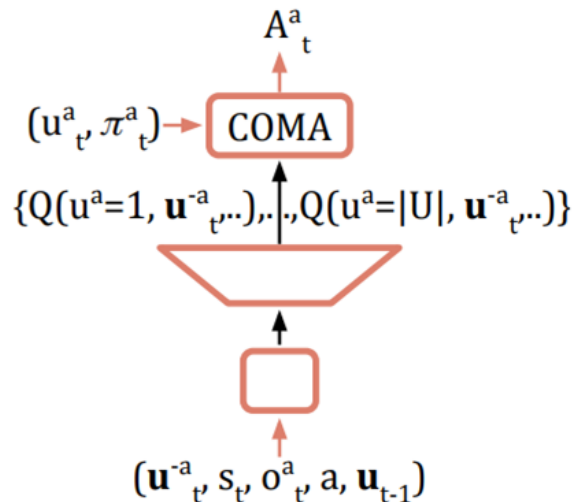
COMA вычисляет функцию преимуществ которая сравнивает Q-value для текущего действия u^a с контрфактическим базисом, который маргинализует (усредняет) действия агента, сохраняя действия других агентов u^{-a} фиксированными.

$$A^a(s, \mathbf{u}) = Q(s, \mathbf{u}) - \underbrace{\sum_{u'^a} \pi^a(u'^a | \tau^a) Q(s, (\mathbf{u}^{-a}, u'^a))}_{\text{counterfactual baseline}}$$

Функция преимуществ $A^a(s, u)$ вычисляет отдельный базис, который использует централизованного критика, чтобы найти контрфакты, когда изменяются только действия агента a , полученное непосредственно из опыта агентов.

Первый член в уравнении представляет собой глобальное Q-значение для текущего выбранного действия, которое указывает на оценку Q-значения в централизованном критике. Второй член в уравнении представляет собой ожидание глобального Q-значения, которое может быть получено при всех возможных действиях агента a . Разница между двумя членами отражает преимущество действия, выбранного текущим агентом, над средним результатом.

COMA



По сравнению с исходным алгоритмом «актор-критик» с одним агентом, градиент политики COMA для всех политик агентов использует указанное выше контрфактуальное преимущество:

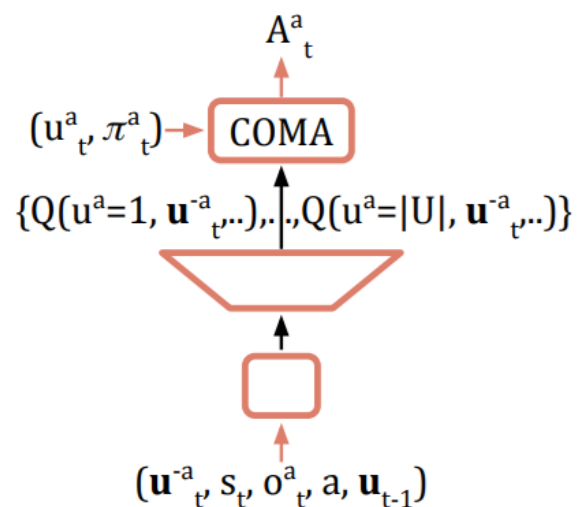
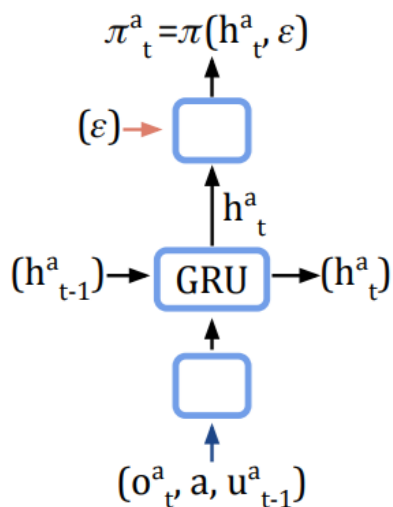
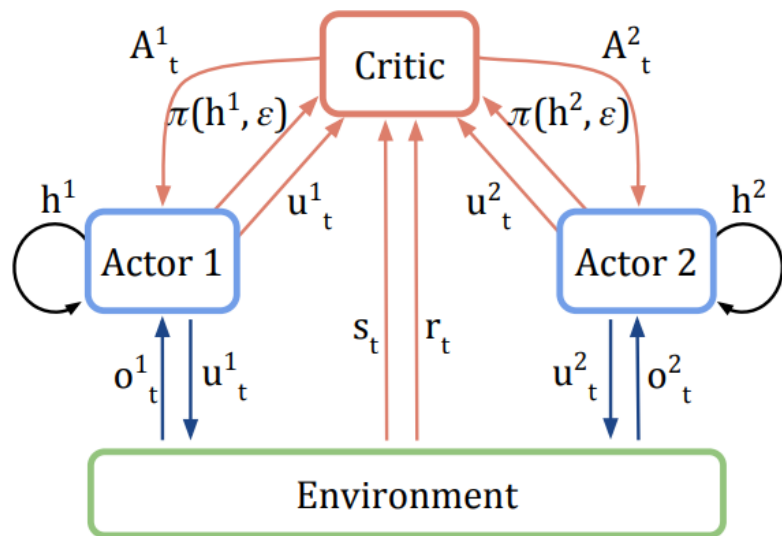
$$g_k = \mathbb{E} \pi \left[\sum_a \nabla_{\theta_k} \log \pi^a(u^a | \tau^a) A^a(s, \mathbf{u}) \right]$$

Каждый агент учится на сформированном вознаграждении, которое сравнивает глобальное вознаграждение с вознаграждением, полученным, когда действие этого агента заменяется действием по умолчанию.

Критик используется только во время обучения и удаляется во время тестирования. Можем рассматривать критика как “обучающий инструмент” алгоритма.

COMA

Counterfactual Multi-Agent Policy Gradients



MADDPG

MADDPG

Рассмотрим смешанную игру, где функции наград агентов произвольны. На «центральном сервере» будем хранить общую оценочную функцию, которая теперь должна выдавать будущие награды сразу для всех агентов.

Определение 126: При данных политиках $\pi^j(a_j \mid s)$ *централизованной оценочной функцией* (centralized state-action value function) обозначим $\vec{Q}(s, \vec{a})$, возвращающую вектор, i -ая компонента которого $Q^i(s, \vec{a})$ равна будущей кумулятивной награде, которую получит i -ый агент после выполнения набора действий $\vec{a}$ в состоянии s .

На центральном сервере понятно, как учить такую Q-функцию, причём это можно делать в off-policy режиме. Для произвольного перехода из буфера $\mathbb{T} := (s, \vec{a}, \vec{r}, s')$ строим таргет

$$y(\mathbb{T}) := \vec{r} + \gamma \vec{Q}(s', \vec{a}'),$$

где $\vec{a}'$ порождены оцениваемыми $\pi^j(a'_j \mid s')$ (для стабилизации процесса — таргет-сетями для них), и минимизируем стандартное MSE:

$$\mathbb{E}_{\mathbb{T}}(\vec{Q}(s, \vec{a}) - y(\mathbb{T}))^2 \rightarrow \min_{\vec{Q}}$$

MADDPG

Процесс необходимо проводить на центральном сервере, поскольку для вычисления таргета необходимы текущие стратегии всех агентов. Вообще, от этого ограничения можно избавиться, если агентам в их наблюдениях доступны действия всех остальных агентов. Тогда мы сможем перейти к децентрализованному обучению следующим образом: i -ый агент может локально хранить и обучать лишь интересующую его компоненту $Q^i(s, \vec{a})$. При построении таргета нужны стратегии оппонентов; их i -ый агент будет *моделировать*. Для моделирования собираются наблюдения (s, a^{-i}) , то есть какие действия предприняли остальные агенты в состоянии s , и дальше агент учится предсказывать действия других игроков по этой обучающей выборке в supervised-режиме:

$$\mathbb{E}_{s, a^{-i}} \log \mu(a^{-i} \mid s) \rightarrow \max_{\mu},$$

обычно с добавлением энтропийного регуляризатора. Имея такое приближение стратегий оппонентов на руках, таргет для обучения критика рассчитывается по формуле

$$y^i(\mathbb{T}) := r^i + Q^i(s', a'^i, a'^{-i}),$$

где $a'^i \sim \pi^i(a'^i \mid s')$, а $a'^{-i} \sim \mu(a'^{-i} \mid s')$.

MADDPG

Как использовать централизованную оценочную функцию для обучения актёров? Для обучения i -го актёра с параметрами θ^i можно применить формулу policy gradient:

$$\nabla_{\theta^i} = \mathbb{E}_s \mathbb{E}_{a^i \sim \pi^i(a^i | s)} \nabla_{\theta^i} \log \pi^i(a^i | s) Q^i(s, a^i, a^{-i}) \quad (8.44)$$

Действительно, для формулы градиентов по θ^i остальные стратегии можно считать фиксированными; поэтому стандартная формула применима. В ней состояния s приходят из опыта взаимодействия агентов со средой, а действия других агентов a^{-i} должны быть порождены этими самими стратегиями прочих агентов.

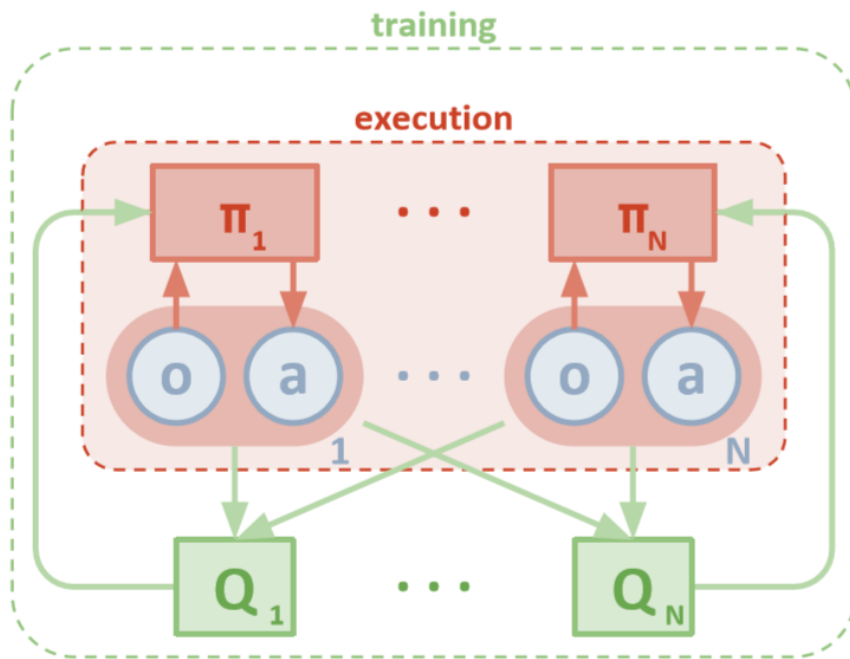
Авторы MADDPG предлагают перейти здесь от policy gradient-схемы к использованию формулы для DDPG, и забыть на частоты посещения состояний. Тогда в формуле (8.44) можно брать s, a^{-i} — из приближения стратегий других агентов. По аналогии с DDPG также можно обучать детерминированную стратегию $\pi^i(s)$ с параметрами θ^i , оптимизируя

$$\mathbb{E}_s \mathbb{E}_{a^i} Q^i(s, \pi^i(s), a^{-i}) \rightarrow \max_{\theta^i},$$

где s берутся из реплей буфера, a^{-i} моделируются или, в случае CTDE-обучения, берутся из честных стратегий $\pi^j(s), j \neq i$; в последнем случае мат.ожидание по a^{-i} вырождается.

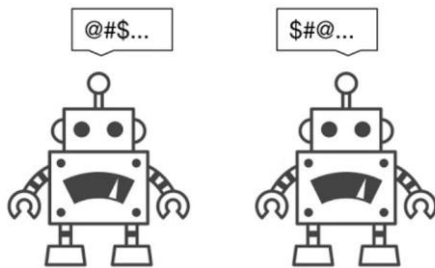
MADDPG

Multi-Agent Deep Deterministic Policy Gradient



Обучение коммуникации

Скажем, что у агента i есть канал связи (communication channel), если на каждом шаге агент помимо действия a_i выбирает некоторое сообщение (message) m_i , которое не влияет на функцию переходов и функции награды, и которое все агенты получают вместе со своими наблюдениями на следующем шаге.

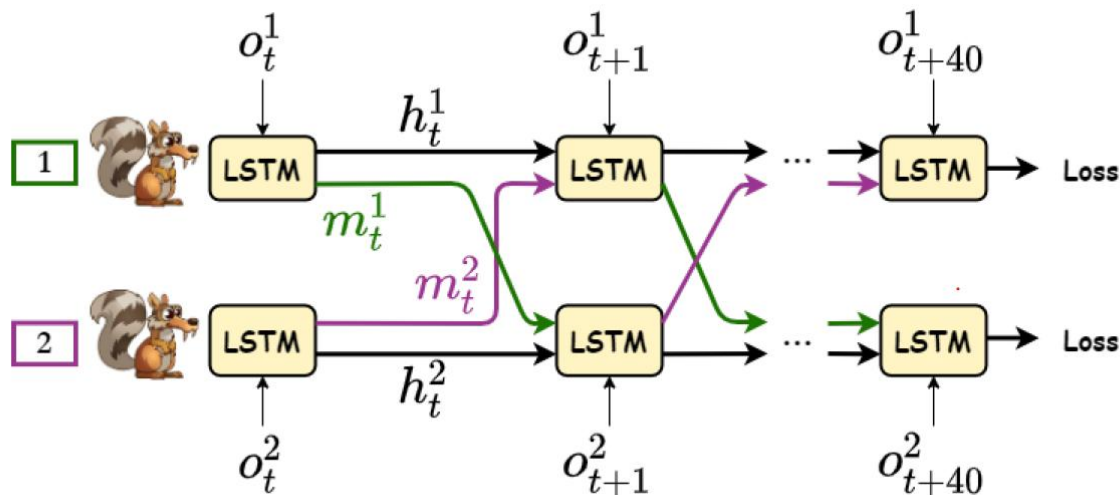


В таком подходе удобно выбрать дискретное пространство сообщений, а при моделировании Q-функции вместо того, чтобы выдавать оценку для каждой пары действий (a_i, m_i) отдельно выдаются оценки для a_i и отдельно для m_i , дальше агент ϵ -жадно выбирает действие для среды и отдельно ϵ -жадно выбирает сообщение для отправки. Это небольшая декомпозиция упрощает пространство действий.

Differentiable Inter-Agent Learning (DIAL)

Для кооперативных задач (с единой функцией награды) в парадигме CTDE.

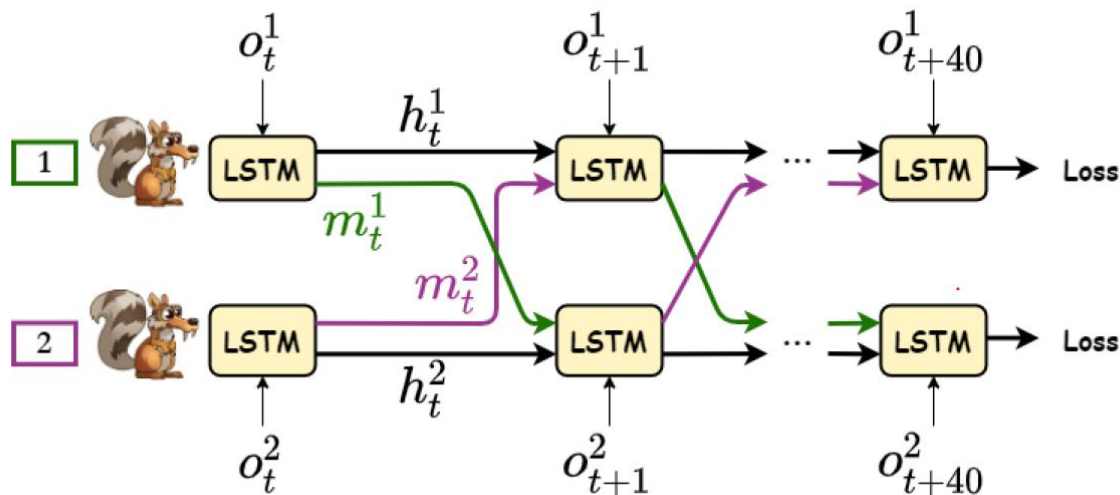
Здесь естественно выбрать непрерывное пространство сообщений. На центральном сервере во время обучения просто заметим, что любые модели агента j , на шаге t минимизирующие свои стандартные функции потерь, дифференцируемы по входным наблюдениям и в том числе по полученным от других агентов сообщениям m_{t-1}^i . Эту информацию о градиенте можно использовать для обучения i -го агента выдавать хорошие сообщения.



Differentiable Inter-Agent Learning (DIAL)

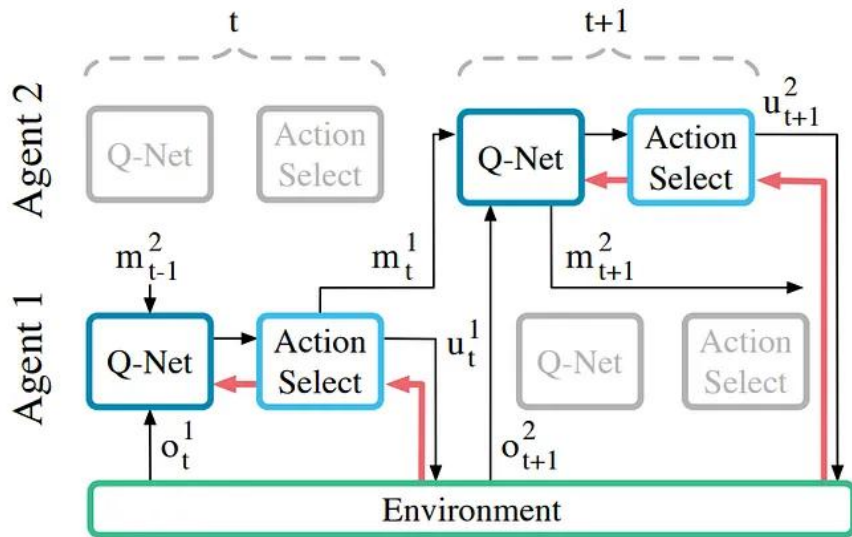
o_t^i — наблюдение, поступающее на вход i -му агенту на шаге t . Оно обрабатывается рекуррентной сетью, которое выдаёт, помимо функций ценности и вероятностей действий, скрытое состояние h_t^i для передачи самому себе на следующий шаг и сообщение m_t^i для передачи другим агентам на следующий шаг.

Далее это сообщение поступает на вход моделям всех других агентов на шаге $t + 1$. В конце вычислений вычисляется некоторая функция потерь в зависимости от используемого алгоритма, и этот градиент проходит не только назад по времени, но и по каналам связи в модели других агентов: как видно, вся эта схема end-to-end дифференцируема.

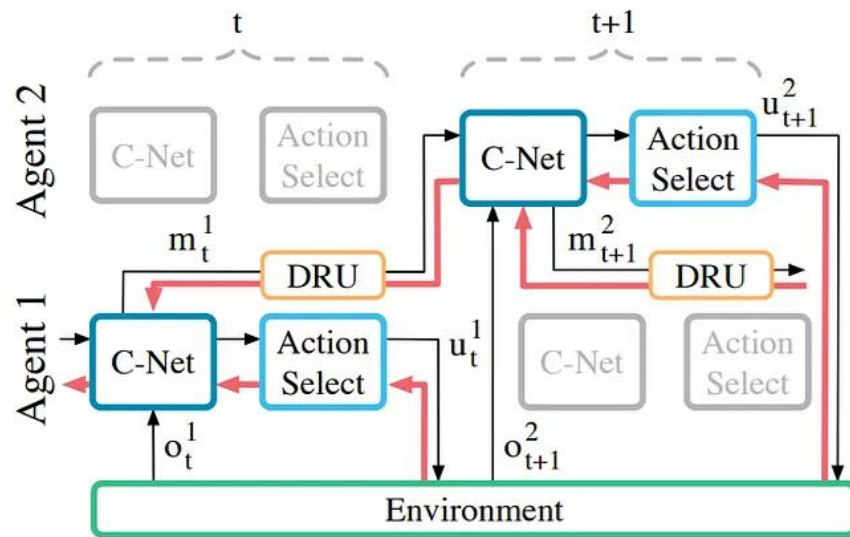


Обучение коммуникации

Reinforced / Differentiable Inter-Agent Learning



(a) RIAL - RL based communication



(b) DIAL - Differentiable communication

In RIAL (a), all Q-values are fed to the action selector, which selects both environment and communication actions. Gradients, shown in red, are computed using DQN for the selected action and flow only through the Q-network of a single agent. In DIAL (b), the message $m_{a \rightarrow b}^t$ bypasses the action selector and instead is processed by the DRU (Section 5.2) and passed as a continuous value to the next C-network. Hence, gradients flow across agents, from the recipient to the sender. For simplicity, at each time step only one agent is highlighted, while the other agent is greyed out.

Рефлексия

Ключевые тезисы

Подведем итоги

1. Многоагентный подход позволяет более точно моделировать сложные среды, где необходимо выполнять управление / планирование большим числом агентов
2. Ожидаемо, что усложнение модели среды и увеличение количества обучаемых агентов порождает новые трудности в процессе обучения, которые необходимо нивелировать
3. В случае моделирования поведения нескольких агентов, особенно при кооперативном / конкурентном взаимодействии надо помнить про эффекты, описываемые в Теории игр
4. Наиболее распространены алгоритмы, использующие CTDE-схему обучения, к ним относятся: QMIX, COMA, MADDPG и т.д.
5. Многоагентный подход ещё больше приближает нас к реальному миру, в котором одновременно функционирует огромное количество "когнитивных агентов" и для эффективного достижения своих целей агенты вынуждены кооперироваться, конкурировать, строить модели других агентов для предсказания их поведения и налаживать коммуникацию

Список материалов для изучения

1. [QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning](#)
2. [COMA: Counterfactual Multi-Agent Policy Gradients](#)
3. [MADDPG: Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments](#)
4. [Алгоритм MADDPG OpenAI](#)
5. [Learning to Communicate with Deep Multi-Agent Reinforcement Learning](#)
6. [Классификация задач мультиагентного обучения с подкреплением](#)
7. Книга: [Мультиагентное обучение с подкреплением \(Алфимцев А.Н.\)](#)
8. [Много-агентное планирование траекторий в децентрализованном режиме: эвристический поиск и обучение с подкреплением](#)
9. [Multi-Agent Reinforcement Learning Framework in SDN-IoT for Transient Load Detection and Prevention](#)
10. [DRLE: Decentralized Reinforcement Learning at the Edge for Traffic Light Control in the IoV](#)
11. [OPTIMIZING TRADING STRATEGIES IN QUANTITATIVE MARKETS USING MULTI-AGENT REINFORCEMENT LEARNING](#)

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет



**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

Спасибо за внимание!